GMS — Gadget Management System

Technical Design Document

Project: GMS (Gadget Management System)
Version: 1.0
Date: April 1, 2026
Tech Stack: Python 3.12, AWS CDK, Serverless
Region: ap-southeast-1

Version History

Date	Version	Description
April 1, 2026	1.0	Initial document creation

Table of Contents

1. System Architecture
2. System End-to-End Flows
3. Backend Lambda Logic
4. Table Design
5. API Gateway Routes
6. CDK Stacks & Deployment
7. Shared Utilities & Layers
8. Appendix

---

1. System Architecture

1.1 Overview

GMS is an AWS serverless backend for managing the full IT asset lifecycle: procurement, AI-powered invoice scanning, approval workflows, employee assignment, handover form generation, issue reporting & repair, software installation governance, asset returns, and disposal with finance notification.

1.2 AWS Services Used

Service	Purpose
AWS Lambda (Python 3.12)	86 functions across all feature domains
API Gateway (REST)	100+ authenticated endpoints
API Gateway (WebSocket)	Real-time notification delivery
DynamoDB	Single-table design with 14 GSIs
S3	Invoice/photo storage, PDF generation, evidence uploads
Cognito	User authentication with 4 role groups
SQS	Email notification queue, finance notification queue
DynamoDB Streams	Event-driven processing (maintenance, notifications, counters)
SES	Transactional email delivery
Textract	AI-powered invoice OCR scanning
SSM Parameter Store	Cross-stack references, configuration
X-Ray	Distributed tracing on all Lambdas
CloudWatch	Logging, metrics, alarms (5XX, 4XX, p99 latency)
SNS	Alarm notifications
CloudFront	Frontend hosting
CodePipeline / CodeBuild	CI/CD for frontend deployment
ECR	Docker image hosting for PDF/Scan Lambdas

1.3 Authentication & Authorization

Authentication is handled by Amazon Cognito User Pool with four role-based groups:

Role	Group Name	Capabilities
IT Admin	`it-admin`	Asset CRUD, assignment, issue management, returns, disposals
Management	`management`	Approval workflows (assets, replacements, software, disposals)
Employee	`employee`	View assigned assets, submit issues, accept handovers, sign returns
Finance	`finance`	View disposal write-off notifications, financial reports

All API Gateway routes use a Cognito User Pools Authorizer. Lambda handlers validate group membership via `require_group()` or `require_roles()` from the shared `auth.py` utility.

1.4 Cross-Stack Reference Pattern

All cross-stack references use SSM Parameter Store — never CloudFormation exports.

Producer stack stores a value:

```
ssm.StringParameter(self, "Id",
    parameter_name=f"/{project}/{env}/layers/base-arn",
    string_value=layer.layer_version_arn)
```

Consumer stack looks it up:

```
arn = ssm.StringParameter.value_for_string_parameter(
    self, f"/{project}/{env}/layers/base-arn")
```

SSM path convention: `/{project}/{env}/{category}/{name}`

1.5 Naming Conventions

All resource names use `helpers/naming.py`:

Resource	Format	Example
Stack	`{project}-{env}-{purpose}-stack`	`gms-production-api-stack`
Lambda	`{project}-{env}-{function-purpose}`	`gms-production-create-asset`
Lambda dir	PascalCase	`CreateAsset/`
DynamoDB Table	`{project}-{env}-{table}`	`gms-production-assets`
S3 Bucket	`{project}-{env}-{purpose}-{account-id}`	`gms-production-assets-562280272590`
SQS Queue	`{project}-{env}-{purpose}`	`gms-production-email-notification-queue`
IAM Role	`{project}-{env}-{purpose}-role`	`gms-production-create-asset-role`
Lambda Layer	`{project}-{env}-{name}-layer`	`gms-production-shared-layer`

---

2. System End-to-End Flows

2.1 Asset Creation & Approval Flow

Lambda Functions: GenerateUploadUrls → ScanWorker → ScanResultProcessor → GetScanResults → CreateAsset → ApproveAsset

1. IT Admin uploads invoice PDF + gadget photos via `POST /assets/uploads` → generates pre-signed S3 URLs and creates an UploadSession record (TTL: 1 hour).
2. `ScanWorker` (Docker Lambda) triggers Textract async analysis on the invoice.
3. `ScanResultProcessor` (SNS-triggered) processes Textract results, extracts fields with confidence scores, updates ScanJob record.
4. IT Admin reviews extracted data via `GET /assets/scan/{scan_job_id}`, confirms/edits fields.
5. `POST /assets` creates the asset record with status `ASSET_PENDING_APPROVAL`, generates a structured Asset ID (`{CATEGORY}-{YEAR}-{NNN}`), validates serial number uniqueness via SerialNumberIndex GSI, and writes an audit log entry.

6. Management approves/rejects via `PUT /assets/{asset_id}/approve`. On approval, status transitions to `IN_STOCK`.

2.2 Asset Assignment & Handover Flow

Lambda Functions: AssignAsset → GetHandoverForm → GenerateSignatureUploadUrl → AcceptHandover

1. IT Admin assigns asset to employee via `POST /assets/{asset_id}/assign`. Creates a HandoverRecord, updates asset status to `ASSIGNED`, sets EmployeeAssetIndex GSI keys.
2. `GetHandoverForm` (Docker Lambda using WeasyPrint) generates a PDF handover form from an HTML template with asset/employee details.
3. Employee uploads digital signature via pre-signed URL from `POST /assets/{asset_id}/signature-upload-url`.
4. Employee accepts handover via `PUT /assets/{asset_id}/accept`. Composites signature onto the PDF, stores signed form in S3.

2.3 Issue Reporting & Repair Flow

Lambda Functions: SubmitIssue → ResolveRepair / SendWarranty / CompleteRepair / RequestReplacement → ManagementReviewIssue

1. Employee submits issue via `POST /assets/{asset_id}/issues`. Validates employee is assigned to the asset. Creates IssueRepairModel with status `TROUBLESHOOTING`. Sends email notification to IT Admins via unified SQS queue.
2. IT Admin triages the issue:
3. **Resolve directly:** `PUT .../resolve-repair` → status `RESOLVED`
4. **Send to warranty:** `PUT .../send-warranty` → status `SEND_WARRANTY`
5. **Complete repair:** `PUT .../complete-repair` → status `RESOLVED`
6. **Request replacement:** `PUT .../request-replacement` → status `REPLACEMENT_REQUIRED`, sends email to Management
7. Management reviews replacement request via `PUT .../management-review` → `REPLACEMENT_APPROVED` or `REPLACEMENT_REJECTED`.

2.4 Software Installation Governance Flow

Lambda Functions: SubmitSoftwareRequest → ReviewSoftwareRequest → ManagementReviewSoftwareRequest

1. Employee submits software request via `POST /assets/{asset_id}/software-requests` with justification, license details, and self-assessed data access impact.
2. IT Admin reviews via `PUT .../review`:
3. **Approve directly** (LOW/MEDIUM risk) → `SOFTWARE_INSTALL_APPROVED`
4. **Escalate to Management** (HIGH risk) → `ESCALATED_TO_MANAGEMENT`
5. **Reject** → `SOFTWARE_INSTALL_REJECTED`
6. Management reviews escalated requests via `PUT .../management-review` → approve or reject.

2.5 Asset Return Flow

Lambda Functions: InitiateReturn → GenerateReturnUploadUrls → SubmitAdminReturnEvidence → GenerateReturnSignatureUploadUrl → CompleteReturn

1. IT Admin initiates return via `POST /assets/{asset_id}/returns` with condition assessment, return trigger (RESIGNATION/REPLACEMENT/TRANSFER/IT_RECALL/UPGRADE), and reset status.

2. IT Admin uploads return photos and admin signature via pre-signed URLs.
3. IT Admin submits evidence via `POST .../submit-evidence`. Validates all S3 files exist. Sends email notification to employee.
4. Employee uploads their signature and completes return via `PUT .../complete`. Composites signatures, updates asset status to `IN_STOCK`.

2.6 Asset Disposal Flow

Lambda Functions: InitiateDisposal → ManagementReviewDisposal → CompleteDisposal → ProcessFinanceNotification

1. IT Admin initiates disposal via `POST /assets/{asset_id}/disposals`. Captures asset specs snapshot. Status → `DISPOSAL_REVIEW`. Sends email to Management.
2. Management reviews via `PUT .../management-review`. On approval → `DISPOSAL_PENDING`. Sends email to IT Admins.
3. IT Admin completes disposal via `PUT .../complete`. Confirms data wipe. Status → `DISPOSED`. Locks both asset and disposal records permanently (`IsLocked = true`). Queues finance notification to SQS.
4. `ProcessFinanceNotification` (SQS consumer) queries all finance-role users, writes FinanceNotificationRecord per recipient, sends SES emails with financial details.

2.7 Email Notification Pipeline

Lambda Functions: (Business Lambda) → SQS Queue → ProcessEmailNotification

All business Lambdas send email events to a unified SQS queue via `send_email_event()`. They never call SES directly.

```
send_email_event(
    Email_Event_Type_Enum.ISSUE_SUBMITTED,
    asset_id=asset_id,
    actor_name=employee_name,
    issue_description=request.issue_description,
)
```

The `ProcessEmailNotification` Lambda consumes the queue, resolves recipients by role from DynamoDB, and sends SES emails.

Supported event types:

Event Type	Recipients	Trigger
`ISSUE_SUBMITTED`	IT Admins	Employee submits issue
`REPLACEMENT_REQUESTED`	Management	IT Admin requests replacement
`SOFTWARE_REQUEST_SUBMITTED`	IT Admins	Employee submits software request
`RETURN_EVIDENCE_SUBMITTED`	Specific employee	IT Admin uploads return evidence
`DISPOSAL_PENDING`	Management	IT Admin initiates disposal
`DISPOSAL_MANAGEMENT_APPROVED`	IT Admins	Management approves disposal

2.8 Real-Time Notification Flow

Lambda Functions: NotificationProcessor (DynamoDB Stream) → WebSocket API → Client

1. DynamoDB Streams trigger `NotificationProcessor` on relevant record changes.
2. Processor creates NotificationRecord with TTL-based expiry.
3. Pushes real-time notification via WebSocket API to connected clients.
4. Clients fetch notification list via `GET /notifications` and mark read via `PATCH /notifications/{id}`.

2.9 Dashboard & Counter Flow

Lambda Functions: CounterProcessor (DynamoDB Stream) → Dashboard Lambdas

1. `CounterProcessor` listens to DynamoDB Streams and maintains pre-computed counters:
2. `DASHBOARD_COUNTERS` — system-wide stats (TotalActiveAssets, PendingIssues, etc.)
3. `ENTITY_COUNTERS` — entity counts (AssetCount, IssueCount, etc.)
4. Per-user counters (AssignedAssets, PendingRequests, PendingSignatures)
5. Dashboard Lambdas read pre-computed counters for O(1) dashboard rendering.

---

3. Backend Lambda Logic

3.1 Asset Management Lambdas

CreateAsset

Validates IT Admin role. Fetches S3 keys from ScanJob → UploadSession chain. Validates uploaded files exist in S3. Generates structured asset ID (`{CATEGORY}-{YEAR}-{NNN}`) via atomic counter. Checks serial number uniqueness via SerialNumberIndex GSI. Creates AssetMetadataModel + AuditLogModel records.

Request	Response
`POST /assets` body: `{ scan_job_id, category, invoice_number, vendor, serial_number, brand, model_name, cost, ... }`	`{ asset_id, status: "ASSET_PENDING_APPROVAL" }`

ApproveAsset

Validates Management role. Uses `transact_write_items` to atomically update asset status and write audit log. On approval: `ASSET_PENDING_APPROVAL` → `IN_STOCK`. On rejection: `ASSET_PENDING_APPROVAL` → `ASSET_REJECTED` with rejection reason.

ListAssets

Supports cursor-based pagination (page size: 20). Queries EntityTypeIndex GSI (`EntityType = "ASSET"`). Optional status filter uses StatusIndex GSI instead. Resolves user IDs to display names via batch lookup.

GetAsset

Fetches asset metadata by PK/SK. Generates pre-signed S3 URLs for invoice and gadget photos. Returns full asset details including lifecycle status.

GetAssetLogs

Queries all `LOG#` sort key records under the asset PK. Batch-resolves actor IDs to names. Returns chronological audit trail.

3.2 Handover & Assignment Lambdas

AssignAsset

Validates IT Admin role. Verifies asset is `IN_STOCK`. Creates HandoverRecord with employee details. Updates asset status to `ASSIGNED`. Sets EmployeeAssetIndex GSI keys for employee → asset lookup.

GetHandoverForm

Docker Lambda (WeasyPrint). Generates PDF handover form from HTML template. Includes asset specs, employee details, company branding. Stores PDF in S3 at `handovers/{asset_id}/{timestamp}.pdf`.

AcceptHandover

Validates Employee role. Verifies caller is the assigned employee. Validates signature file exists in S3. Composites signature onto handover PDF. Updates HandoverRecord with `AcceptedAt` timestamp.

CancelAssignment

Validates IT Admin role. Verifies handover has not been accepted. Removes EmployeeAssetIndex GSI keys. Reverts asset status to `IN_STOCK`.

3.3 Issue Management Lambdas

SubmitIssue

Validates Employee role. Verifies employee is assigned to the asset via latest HandoverRecord. Generates domain-prefixed ID (`ISSUE-{YYYYMM}-{N}`). Uses `transact_write_items` to atomically create IssueRepairModel + update asset status to `ISSUE_REPORTED`. Backfills MaintenanceEntityIndex GSI fields. Queues email notification to IT Admins.

ResolveRepair

IT Admin marks issue as resolved directly. Status: `TROUBLESHOOTING` → `RESOLVED`. Sets `ActionPath = "REPAIR"`.

SendWarranty

IT Admin sends asset to warranty. Status: `TROUBLESHOOTING` → `SEND_WARRANTY`. Records warranty notes.

CompleteRepair

IT Admin completes repair after warranty/repair. Status: `UNDER_REPAIR` / `SEND_WARRANTY` → `RESOLVED`. Records completion notes.

RequestReplacement

IT Admin requests asset replacement. Status → `REPLACEMENT_REQUIRED`. Queues email to Management.

ManagementReviewIssue

Management approves/rejects replacement. Status → `REPLACEMENT_APPROVED` or `REPLACEMENT_REJECTED`.

ListIssues

Lists issues for a specific asset. Cursor-based pagination on main table (`SK begins_with "ISSUE#"`).

ListAllIssues

Lists all issues across all assets. Queries IssueEntityIndex GSI (`IssueEntityType = "ISSUE"`). Supports status filtering via IssueStatusIndex GSI.

3.4 Software Governance Lambdas

SubmitSoftwareRequest

Validates Employee role. Generates domain-prefixed ID (`SOFTWARE-{YYYYMM}-{N}`). Creates SoftwareInstallationModel with status `PENDING_REVIEW`. Queues email to IT Admins.

ReviewSoftwareRequest

IT Admin reviews request. Sets risk level (LOW/MEDIUM/HIGH). LOW/MEDIUM → approve directly. HIGH → escalate to Management. Or reject with reason.

ManagementReviewSoftwareRequest

Management reviews escalated requests. Approve or reject with remarks.

3.5 Return Lambdas

InitiateReturn

IT Admin initiates return. Validates asset is `ASSIGNED`. Creates ReturnRecordModel with condition assessment, return trigger, and reset status. Captures asset snapshot (SerialNumber, Model). Status → `RETURN_PENDING`.

SubmitAdminReturnEvidence

IT Admin submits return photos + admin signature. Validates all S3 files exist (with DynamoDB cleanup for stale keys). Queues email to employee.

CompleteReturn

Employee signs and completes return. Validates employee signature exists. Updates asset status to `IN_STOCK`. Clears EmployeeAssetIndex GSI keys.

3.6 Disposal Lambdas

InitiateDisposal

IT Admin initiates disposal. Captures AssetSpecs snapshot. Creates DisposalRecordModel. Asset status → `DISPOSAL_REVIEW`. Queues email to Management.

ManagementReviewDisposal

Management approves/rejects disposal. On approval → `DISPOSAL_PENDING`. On rejection → `DISPOSAL_REJECTED`.

CompleteDisposal

IT Admin completes disposal. Confirms data wipe. Uses `transact_write_items` to atomically: lock asset record, lock disposal record, set asset status to `DISPOSED`. Queues finance notification to SQS.

ProcessFinanceNotification

SQS consumer. Queries all finance-role users from DynamoDB. Creates FinanceNotificationRecord per recipient. Sends SES email with financial details (original cost, disposal date, reason).

3.7 Upload & Scan Lambdas

GenerateUploadUrls

Generates pre-signed S3 PUT URLs for invoice + gadget photos. Creates UploadSession record with TTL (1 hour auto-cleanup). Triggers ScanWorker.

ScanWorker

Docker Lambda. Starts Textract async document analysis on uploaded invoice. Creates ScanJob record with status `PROCESSING`.

ScanResultProcessor

SNS-triggered by Textract completion. Extracts fields with confidence scores (InvoiceNumber, Vendor, SerialNumber, Brand, Model, Cost, etc.). Updates ScanJob with extracted data. Status → `COMPLETED` or `SCAN_FAILED`.

3.8 User Management Lambdas

CreateUser

IT Admin creates user in Cognito + DynamoDB. Sets role group membership. Creates UserMetadataModel with initial counters.

DeactivateUser

IT Admin deactivates user. Disables Cognito account. Updates DynamoDB status to `inactive`.

ListUsers

Queries EntityTypeIndex GSI (`EntityType = "USER"`). Cursor-based pagination.

3.9 Notification Lambdas

ListMyNotifications

Returns notifications for the authenticated user. Queries by `PK = USER#{actor_id}`, `SK begins_with "NOTIFICATION#"`. Cursor-based pagination.

MarkNotificationRead

Updates `IsRead = true` on a specific notification record. Decrements user's `UnreadNotificationCount`.

3.10 Dashboard Lambdas

GetDashboardCounters

Reads pre-computed `DASHBOARD_COUNTERS` record. Returns TotalActiveAssets, PendingIssues, PendingApprovals, etc.

GetITAdminStats / GetManagementStats / GetEmployeeStats / GetFinanceStats

Role-specific dashboard statistics. Read from pre-computed counter records.

GetAssetDistribution

Returns asset count by category from pre-computed `CategoryCounts` map.

GetRecentActivity

Queries ActivityEntityIndex GSI for latest activity records across all domains.

GetApprovalHub

Management-specific. Aggregates pending approvals across all domains (assets, replacements, software escalations, disposals).

3.11 Category Management Lambdas

CreateAssetCategory

IT Admin creates a new asset category. Validates uniqueness via CategoryNameIndex GSI. Stores as `CATEGORY#{uuid}` record.

DeleteAssetCategory

IT Admin deletes a category. Validates no assets are using the category before deletion.

3.12 Stream Processor Lambdas

CounterProcessor

DynamoDB Streams consumer. Maintains pre-computed counters for dashboard rendering. Updates DASHBOARD_COUNTERS, ENTITY_COUNTERS, and per-user counters on relevant record changes.

MaintenanceStreamProcessor

DynamoDB Streams consumer. Processes maintenance-related record changes for the MaintenanceEntityIndex GSI.

NotificationProcessor

DynamoDB Streams consumer. Creates in-app notification records and pushes real-time updates via WebSocket API.

---

4. Table Design

4.1 DynamoDB Table

Single-table design: `gms-{env}-assets`

- **Billing:** On-demand (PAY_PER_REQUEST)
- **Stream:** NEW_AND_OLD_IMAGES
- **TTL attribute:** `TTL`
- **Partition Key:** `PK` (String)
- **Sort Key:** `SK` (String)

4.2 Record Types

Record Type	PK Pattern	SK Pattern	Model
Upload Session	`SESSION#<UploadSessionID>`	`METADATA`	UploadSessior
Scan Job	`SCAN#<ScanJobID>`	`METADATA`	ScanJobMode
Asset Metadata	`ASSET#<AssetID>`	`METADATA`	AssetMetadata
Handover Record	`ASSET#<AssetID>`	`HANDOVER#<HandoverID>`	HandoverRecc
Audit Log	`ASSET#<AssetID>`	`LOG#<Timestamp>#<ActorID>`	AuditLogMode
Software Request	`ASSET#<AssetID>`	`SOFTWARE#<SoftwareRequestID>`	SoftwareInstal
Issue/ Repair	`ASSET#<AssetID>`	`ISSUE#<IssueID>`	IssueRepairMo
Disposal Record	`ASSET#<AssetID>`	`DISPOSAL#<DisposalID>`	DisposalRecor
Return Record	`ASSET#<AssetID>`	`RETURN#<ReturnID>`	ReturnRecordI
User Metadata	`USER#<UserID>`	`METADATA`	UserMetadatal
Finance Notification	`ASSET#<AssetID>`	`FINANCE_NOTIFICATION#<NotifiedAt>#<RecipientUserID>`	FinanceNotific
Asset Category	`CATEGORY#<CategoryID>`	`METADATA`	AssetCategory
Category Counter	`COUNTER#<Category>#<Year>`	`METADATA`	CategoryCoun

Record Type	PK Pattern	SK Pattern	Model
Domain Counter	`DOMAIN_COUNTER#<Domain>#<YYYYMM>`	`METADATA`	DomainCounte
Entity Counter	`ENTITY_COUNTERS`	`METADATA`	EntityCounterM
Dashboard Counter	`DASHBOARD_COUNTERS`	`METADATA`	DashboardCou
Notification	`USER#<RecipientUserID>`	`NOTIFICATION#<Timestamp>#<NotificationID>`	NotificationRec
Activity	`ACTIVITY#<ActivityID>`	`METADATA`	ActivityRecord

4.3 Global Secondary Indexes (14 GSIs)

#	GSI Name	PK Field	SK Field	Used By
1	EntityTypeIndex	`EntityType` ("ASSET"/"USER")	`CreatedAt`	ListAssets, ListUsers
2	StatusIndex	`StatusIndexPK` (`STATUS#<Status>`)	`StatusIndexSK` (`ASSET#<AssetID>`)	ListAssets (with status filter)
3	SerialNumberIndex	`SerialNumberIndexPK` (`SERIAL#<SN>`)	`SerialNumberIndexSK`	Serial number uniqueness check
4	EmployeeAssetIndex	`EmployeeAssetIndexPK` (`EMPLOYEE#<EmpID>`)	`EmployeeAssetIndexSK` (`ASSET#<Date>`)	ListMyAssets, ListEmployeeSignatures
5	SoftwareStatusIndex	`SoftwareStatusIndexPK` (`SOFTWARE_STATUS#<Status>`)	`SoftwareStatusIndexSK` (`SOFTWARE#<CreatedAt>`)	ListEscalatedRequests
6	IssueStatusIndex	`IssueStatusIndexPK` (`ISSUE_STATUS#<Status>`)	`IssueStatusIndexSK` (`ISSUE#<CreatedAt>`)	ListPendingReplacements
7	DisposalStatusIndex	`DisposalStatusIndexPK` (`DISPOSAL_STATUS#<Status>`)	`DisposalStatusIndexSK` (`DISPOSAL#<DisposalID>`)	ListPendingDisposals
8	DisposalEntityIndex	`DisposalEntityType` ("DISPOSAL")	`InitiatedAt`	ListDisposals (all)
9	IssueEntityIndex	`IssueEntityType` ("ISSUE")	`CreatedAt`	ListAllIssues
10	SoftwareEntityIndex	`SoftwareEntityType` ("SOFTWARE_REQUEST")	`CreatedAt`	ListAllSoftwareRequests
11	MaintenanceEntityIndex	`MaintenanceEntityType` ("MAINTENANCE")	`MaintenanceTimestamp`	ListMaintenanceHistory
12	CategoryEntityIndex	`CategoryEntityType` ("CATEGORY")	`CreatedAt`	ListAssetCategories
13	CategoryNameIndex	`CategoryNameIndexPK` (`CATEGORY_NAME#<Name>`)	— (KEYS_ONLY)	Category uniqueness check
14	ActivityEntityIndex	`ActivityEntityType` ("ACTIVITY")	`Timestamp`	GetRecentActivity

4.4 ID Generation

Asset IDs: `{CATEGORY}-{YEAR}-{NNN}` (e.g., `LAPTOP-2026-001`) - Uses atomic counter: `COUNTER#{category}#{year}` → `Count`

Domain-prefixed IDs: `{DOMAIN}-{YYYYMM}-{N}` (e.g., `ISSUE-202604-1`) - Uses atomic counter: `DOMAIN_COUNTER#{domain}#{YYYYMM}` → `Count` - Domains: ISSUE, RETURN, DISPOSAL, SOFTWARE

Other IDs: UUID v4 for Handover, Scan Job, Upload Session, Category, Activity

4.5 S3 Bucket

Bucket: `gms-{env}-assets-{account-id}`

Prefix	Content	Lifecycle
`uploads/`	Temporary invoice/photo uploads	7-day expiry
`handovers/{asset_id}/`	Generated PDF handover forms	Permanent
`signatures/{employee_id}/{asset_id}/`	Digital signatures	Permanent
`returns/{asset_id}/`	Return evidence photos + signatures	Permanent
`issues/{asset_id}/{issue_id}/`	Issue evidence photos	Permanent

---

5. API Gateway Routes

REST API: `gms-{env}-api-gateway`
Stage: `{env}` (e.g., `production`)
Authorizer: Cognito User Pools on all routes
CORS: All origins, all methods

5.1 Asset Routes

Method	Path	Lambda	Role
POST	`/assets`	CreateAsset	IT Admin
GET	`/assets`	ListAssets	IT Admin, Management
GET	`/assets/{asset_id}`	GetAsset	All roles
GET	`/assets/{asset_id}/logs`	GetAssetLogs	IT Admin, Management
PUT	`/assets/{asset_id}/approve`	ApproveAsset	Management
POST	`/assets/uploads`	GenerateUploadUrls	IT Admin
GET	`/assets/scan/{scan_job_id}`	GetScanResults	IT Admin

5.2 Handover & Assignment Routes

Method	Path	Lambda	Role
POST	`/assets/{asset_id}/assign`	AssignAsset	IT Admin
GET	`/assets/{asset_id}/assign-pdf-form`	GetHandoverForm	IT Admin, Employee
GET	`/assets/{asset_id}/signed-pdf-form`	GetSignedHandoverForm	IT Admin, Employee
POST	`/assets/{asset_id}/signature-upload-url`	GenerateSignatureUploadUrl	Employee
PUT	`/assets/{asset_id}/accept`	AcceptHandover	Employee
DELETE	`/assets/{asset_id}/cancel-assignment`	CancelAssignment	IT Admin

5.3 User Management Routes

Method	Path	Lambda	Role
GET	`/users`	ListUsers	IT Admin
POST	`/users/create`	CreateUser	IT Admin
DELETE	`/users/{id}/deactivate`	DeactivateUser	IT Admin
PUT	`/users/{id}/reactivate`	ReactivateUser	IT Admin
GET	`/users/{id}/signatures`	ListEmployeeSignatures	IT Admin
GET	`/users/me/pending-signatures`	ListPendingSignatures	Employee

5.4 Software Governance Routes

Method	Path	Lambda	Role
POST	`/assets/{asset_id}/software-requests`	SubmitSoftwareRequest	Employee
GET	`/assets/{asset_id}/software-requests`	ListSoftwareRequests	All roles
GET	`/assets/{asset_id}/software-requests/{id}`	GetSoftwareRequest	All roles
PUT	`/assets/{asset_id}/software-requests/{id}/review`	ReviewSoftwareRequest	IT Admin
PUT	`/assets/{asset_id}/software-requests/{id}/management-review`	ManagementReviewSoftwareRequest	Management
GET	`/software-requests`	ListAllSoftwareRequests	IT Admin

5.5 Issue Management Routes

Method	Path	Lambda	Role
POST	`/assets/{asset_id}/issues`	SubmitIssue	Employee
GET	`/assets/{asset_id}/issues`	ListIssues	IT Admin, Employee

Method	Path	Lambda	Role
GET	`/assets/{asset_id}/issues/{id}`	GetIssue	All roles
POST	`/assets/{asset_id}/issues/{id}/upload-urls`	GenerateIssueUploadUrls	Employee
PUT	`/assets/{asset_id}/issues/{id}/resolve-repair`	ResolveRepair	IT Admin
PUT	`/assets/{asset_id}/issues/{id}/send-warranty`	SendWarranty	IT Admin
PUT	`/assets/{asset_id}/issues/{id}/complete-repair`	CompleteRepair	IT Admin
PUT	`/assets/{asset_id}/issues/{id}/request-replacement`	RequestReplacement	IT Admin
PUT	`/assets/{asset_id}/issues/{id}/management-review`	ManagementReviewIssue	Management
GET	`/issues`	ListAllIssues	IT Admin
GET	`/issues/pending-replacements`	ListPendingReplacements	Management

5.6 Return Routes

Method	Path	Lambda	Role
POST	`/assets/{asset_id}/returns`	InitiateReturn	IT Admin
GET	`/assets/{asset_id}/returns`	ListReturns	IT Admin, Management
GET	`/assets/{asset_id}/returns/{id}`	GetReturn	All roles
POST	`/assets/{asset_id}/returns/{id}/upload-urls`	GenerateReturnUploadUrls	IT Admin
POST	`/assets/{asset_id}/returns/{id}/submit-evidence`	SubmitAdminReturnEvidence	IT Admin
POST	`/assets/{asset_id}/returns/{id}/signature-upload-url`	GenerateReturnSignatureUploadUrl	Employee
PUT	`/assets/{asset_id}/returns/{id}/complete`	CompleteReturn	Employee
GET	`/returns`	ListAllReturns	IT Admin

5.7 Disposal Routes

Method	Path	Lambda	Role
POST	`/assets/{asset_id}/disposals`	InitiateDisposal	IT Admin
GET	`/assets/{asset_id}/disposals`	ListAssetDisposals	IT Admin, Management
GET	`/assets/{asset_id}/disposals/{id}`	GetDisposalDetails	All roles
PUT	`/assets/{asset_id}/disposals/{id}/management-review`	ManagementReviewDisposal	Management
PUT	`/assets/{asset_id}/disposals/{id}/complete`	CompleteDisposal	IT Admin

Method	Path	Lambda	Role
GET	`/disposals`	ListDisposals	IT Admin, Management
GET	`/disposals/pending`	ListPendingDisposals	Management

5.8 Notification Routes

Method	Path	Lambda	Role
GET	`/notifications`	ListMyNotifications	All roles
PATCH	`/notifications/{id}`	MarkNotificationRead	All roles

5.9 Category Routes

Method	Path	Lambda	Role
POST	`/categories`	CreateAssetCategory	IT Admin
GET	`/categories`	ListAssetCategories	All roles
DELETE	`/categories/{id}`	DeleteAssetCategory	IT Admin

5.10 Dashboard Routes

Method	Path	Lambda	Role
GET	`/dashboard/counters`	GetDashboardCounters	All roles
GET	`/dashboard/it-admin/stats`	GetITAdminStats	IT Admin
GET	`/dashboard/management/stats`	GetManagementStats	Management
GET	`/dashboard/management/approval-hub`	GetApprovalHub	Management
GET	`/dashboard/employee/stats`	GetEmployeeStats	Employee
GET	`/dashboard/finance/stats`	GetFinanceStats	Finance
GET	`/dashboard/asset-distribution`	GetAssetDistribution	All roles
GET	`/dashboard/recent-activity`	GetRecentActivity	All roles
GET	`/pages/assets/stats`	GetAssetsPageStats	All roles
GET	`/pages/requests/it-admin/stats`	GetRequestsITAdminStats	IT Admin
GET	`/pages/requests/employee/stats`	GetRequestsEmployeeStats	Employee

5.11 CloudWatch Alarms

Alarm	Metric	Threshold	Period
5XX Errors	Server errors	≥ 1	1 min
4XX Errors	Client errors	≥ 50	5 min

Alarm	Metric	Threshold	Period
p99 Latency	Response time	≥ 5000ms	5 min (3 periods)
Integration Latency	Lambda execution	≥ 10000ms	5 min (3 periods)

All alarms notify via SNS → email subscription.

---

6. CDK Stacks & Deployment

6.1 Stack Inventory

#	Stack	Purpose	Key Resources
1	`layers-stack`	Lambda layers	dependencies layer (Docker-bundled), shared layer
2	`storage-stack`	Core data stores	DynamoDB table (14 GSIs), S3 bucket
3	`auth-stack`	Authentication	Cognito User Pool, Post Confirmation trigger
4	`upload-stack`	File uploads	GenerateUploadUrls Lambda
5	`scan-stack`	AI scanning	ScanWorker (Docker), ScanResultProcessor, GetScanResults
6	`asset-stack`	Asset CRUD	CreateAsset, ApproveAsset, ListAssets, GetAsset, GetAssetLogs
7	`user-stack`	User management	CreateUser, DeactivateUser, ReactivateUser, ListUsers
8	`handover-stack`	Handover workflow	AssignAsset (Docker/PDF), AcceptHandover, CancelAssignment
9	`software-governance-stack`	Software requests	Submit, Review, ManagementReview, List Lambdas
10	`issue-management-stack`	Issue lifecycle	14 Lambdas for full issue/repair workflow
11	`issue-events-stack`	Issue stream processing	DynamoDB Streams processor
12	`return-stack`	Asset returns	Initiate, Evidence, Complete, List Lambdas
13	`disposal-stack`	Asset disposal	Initiate, Review, Complete + Finance SQS queue
14	`email-notification-stack`	Unified email pipeline	SQS queue + ProcessEmailNotification Lambda
15	`notification-stack`	In-app notifications	NotificationProcessor, List, MarkRead Lambdas
16	`websocket-stack`	Real-time updates	WebSocket API (Connect, Disconnect, Default)
17	`maintenance-stack`	Maintenance history	MaintenanceStreamProcessor Lambda
18	`category-stack`	Asset categories	Create, Delete, List category Lambdas
19	`counter-stack`	Atomic counters	CounterProcessor (DynamoDB Streams)
20	`dashboard-stack`	Dashboard stats	10 dashboard/stats Lambdas

#	Stack	Purpose	Key Resources
21	`api-stack`	API Gateway	REST API, Cognito authorizer, all route bindings, CloudWatch alarms
22	`frontend-pipeline-stack`	CI/CD	CodePipeline, CodeBuild, CloudFront, S3 hosting

6.2 Deployment Order

Stack dependencies are declared in `app.py` via `stack.add_dependency()`:

```
layers-stack
  └── storage-stack
        ├── auth-stack
        ├── upload-stack ← scan-stack
        ├── scan-stack
        ├── asset-stack
        ├── user-stack
        ├── handover-stack
        ├── email-notification-stack
        │     ├── software-governance-stack
        │     ├── issue-management-stack ← issue-events-stack
        │     ├── return-stack
        │     └── disposal-stack
        ├── notification-stack ← websocket-stack
        ├── maintenance-stack
        ├── category-stack
        ├── counter-stack
        └── dashboard-stack
              └── api-stack (depends on all feature stacks)
                    └── frontend-pipeline-stack
```

6.3 Lambda Layers

Dependencies Layer (Docker-bundled): - aws-lambda-powertools - boto3 / botocore - simplejson - aws-xray-sdk - opensearch-py - requests + requests-aws4auth - pydantic

Shared Layer (direct code): - `custom_exceptions/` — NotFoundException, ConflictException, etc. - `utils/auth.py` — Cognito group validation - `utils/ddb_helper.py` — DynamoDB CRUD + paginated queries - `utils/email_queue.py` — Unified email notification sender - `utils/enums.py` — All status/type enumerations - `utils/id_generator.py` — Domain-prefixed ID generation - `utils/lock.py` — Record lock checking - `utils/models.py` — Pydantic DynamoDB models - `utils/pagination.py` — Cursor-based pagination (PAGE_SIZE = 20) - `utils/response.py` — Standardized success/error responses with CORS - `utils/s3_helper.py` — S3 validation with DynamoDB cleanup - `utils/user_resolver.py` — Batch user name resolution

6.4 Docker Lambdas

Two Lambda functions use Docker images instead of standard layers:

PDF Generation (`services/lambdas/docker/pdf/Dockerfile`): - Base: `public.ecr.aws/lambda/python:3.12` - System deps: pango, gdk-pixbuf2, libffi-devel (for WeasyPrint) - Used by: AssignAsset (handover form generation)

Asset Scanning (`services/lambdas/docker/scan/Dockerfile`): - Base: `public.ecr.aws/lambda/python:3.12` - Used by: ScanWorker (Textract integration)

Both Dockerfiles copy the shared layer code and function code into the image. The `FUNCTION_DIR` build arg is passed by CDK to select the correct Lambda handler.

6.5 Environment Configuration

```
# helpers/environment.py
PROJECT_NAME = "gms"
ENVIRONMENT = "production"  # overridable via DEPLOY_ENV env var
REGION = "ap-southeast-1"
```

CDK context (from `cdk.json`): - `certificate_arn`: ACM certificate for CloudFront - `domain_name`: Custom domain for frontend

6.6 Common CDK Commands

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

cdk ls        # list all stacks
cdk synth     # synthesize CloudFormation templates
cdk diff      # diff against deployed stack
cdk deploy    # deploy all stacks
```

---

7. Shared Utilities & Layers

7.1 Response Format (`utils/response.py`)

All Lambda functions return standardized responses:

Success:

```
{
  "statusCode": 200,
  "headers": {
    "Access-Control-Allow-Origin": "*",
    "Access-Control-Allow-Headers": "Content-Type,Authorization,X-Amz-Date,X-Api-Key",
    "Access-Control-Allow-Methods": "GET,POST,PUT,DELETE,OPTIONS"
  },
  "body": "{ ... }"
}
```

Error:

```
{
  "statusCode": 400,
  "headers": { ... },
  "body": "{ \"message\": \"Error description\" }"
}
```

7.2 Pagination (`utils/pagination.py`)

All list endpoints use cursor-based pagination with a fixed page size of 20.

- Cursors are opaque base64-encoded DynamoDB `LastEvaluatedKey` tokens
- O(1) per page regardless of dataset size
- When `FilterExpression` is used, the paginated_query helper loops internally until 20 items are collected

Query params: `GET /assets?cursor=eyJQSyI6...&sort_order=desc`

Response:

```
{
  "items": [...],
  "count": 20,
  "next_cursor": "eyJQSyI6...",
  "has_next_page": true
}
```

7.3 Authentication (`utils/auth.py`)

Function	Purpose	Returns
`require_group(event, group)`	Validate caller belongs to required group	`actor_id` (sub claim)
`require_roles(event, roles)`	Validate caller belongs to one of allowed roles	`(actor_id, matched_role)`
`get_caller_info(event)`	Extract caller identity	`(actor_id, groups)`

7.4 DynamoDB Helper (`utils/ddb_helper.py`)

Function	Purpose
`get_item(table, key)`	Single item fetch
`put_item(table, item)`	Create/overwrite item
`update_item(table, key, updates)`	Partial update with SET expression
`delete_item(table, key)`	Delete item
`query_index(table, index, key_condition, filter)`	Simple GSI query
`paginated_query(table, index, key_condition, filter, cursor, scan_forward)`	Cursor-based paginated query

7.5 Domain ID Generator (`utils/id_generator.py`)

```
generate_domain_id(table, "ISSUE")  # → "ISSUE-202604-1"
```

Uses atomic DynamoDB counter (`ADD #count :inc` with `ReturnValues="UPDATED_NEW"`).

7.6 Email Queue (`utils/email_queue.py`)

```
send_email_event(Email_Event_Type_Enum.ISSUE_SUBMITTED, asset_id=asset_id, ...)
```

Reads `EMAIL_NOTIFICATION_QUEUE_URL` from environment. Sends JSON message to SQS. Gracefully skips if env var is not set.

7.7 S3 Validation (`utils/s3_helper.py`)

Function	Purpose
`validate_s3_key(bucket, key, label)`	Raise ValueError if file missing
`validate_s3_keys(bucket, keys, label)`	Validate list of S3 keys
`validate_and_clean_s3_key(table, pk, sk, field, bucket, key)`	Validate + remove stale DDB attribute
`validate_and_clean_s3_keys(table, pk, sk, field, bucket, keys)`	Validate list + remove stale DDB attribute

7.8 User Resolver (`utils/user_resolver.py`)

Batch-fetches user fullnames for a set of user IDs via DynamoDB `BatchGetItem`. Supports up to 100 keys per call with automatic pagination for unprocessed keys. Falls back to user ID for missing users.

7.9 Record Lock (`utils/lock.py`)

```
check_record_lock(item, "asset")  # raises ConflictException if IsLocked=True
```

Used by CompleteDisposal to permanently lock asset and disposal records.

7.10 Enums (`utils/enums.py`)

All enums inherit `(str, Enum)` for DynamoDB backward compatibility.

Enum	Values
`Asset_Status_Enum`	IN_STOCK, ASSIGNED, ASSET_PENDING_APPROVAL, ASSET_REJECTED, DISPOSAL_REVIEW, DISPOSAL_PENDING, DISPOSAL_REJECTED, DISPOSED, RETURN_PENDING, DAMAGED, REPAIR_REQUIRED, UNDER_REPAIR, ISSUE_REPORTED
`Issue_Status_Enum`	TROUBLESHOOTING, UNDER_REPAIR, SEND_WARRANTY, RESOLVED, REPLACEMENT_REQUIRED, REPLACEMENT_APPROVED, REPLACEMENT_REJECTED
`Software_Status_Enum`	PENDING_REVIEW, ESCALATED_TO_MANAGEMENT, SOFTWARE_INSTALL_APPROVED, SOFTWARE_INSTALL_REJECTED
`Return_Condition_Enum`	GOOD, MINOR_DAMAGE, MINOR_DAMAGE_REPAIR_REQUIRED, MAJOR_DAMAGE
`Return_Trigger_Enum`	RESIGNATION, REPLACEMENT, TRANSFER, IT_RECALL, UPGRADE
`User_Role_Enum`	it-admin, management, employee, finance

Enum	Values
`Notification_Type_Enum`	22 notification types across all roles
`Email_Event_Type_Enum`	ISSUE_SUBMITTED, REPLACEMENT_REQUESTED, SOFTWARE_REQUEST_SUBMITTED, RETURN_EVIDENCE_SUBMITTED, DISPOSAL_PENDING, DISPOSAL_MANAGEMENT_APPROVED

8. Appendix

8.1 Complete Lambda Function Inventory (86 Functions)

#	Function Directory	Purpose	Trigger
1	AcceptHandover	Employee accepts asset handover	API Gateway
2	ApproveAsset	Management approves/rejects asset	API Gateway
3	AssignAsset	IT Admin assigns asset to employee (Docker/PDF)	API Gateway
4	CancelAssignment	IT Admin cancels pending assignment	API Gateway
5	CognitoPostConfirmation	Auto-create DDB user on Cognito signup	Cognito trigger
6	CompleteDisposal	IT Admin completes disposal + locks records	API Gateway
7	CompleteRepair	IT Admin completes repair after warranty	API Gateway
8	CompleteReturn	Employee signs and completes return	API Gateway
9	CounterProcessor	Maintains pre-computed dashboard counters	DynamoDB Stream
10	CreateAsset	IT Admin creates asset from scan results	API Gateway
11	CreateAssetCategory	IT Admin creates asset category	API Gateway
12	CreateUser	IT Admin creates user in Cognito + DDB	API Gateway
13	DeactivateUser	IT Admin deactivates user account	API Gateway
14	DeleteAssetCategory	IT Admin deletes asset category	API Gateway
15	GenerateIssueUploadUrls	Pre-signed URLs for issue evidence photos	API Gateway
16	GenerateReturnSignatureUploadUrl	Pre-signed URL for employee return signature	API Gateway
17	GenerateReturnUploadUrls	Pre-signed URLs for return evidence photos	API Gateway
18	GenerateSignatureUploadUrl	Pre-signed URL for handover signature	API Gateway
19	GenerateUploadUrls	Pre-signed URLs for invoice + gadget photos	API Gateway
20	GetApprovalHub	Management pending approvals aggregation	API Gateway
21	GetAsset	Fetch asset details with pre-signed S3 URLs	API Gateway
22	GetAssetDistribution	Asset count by category	API Gateway
23	GetAssetLogs	Chronological audit trail for asset	API Gateway
24	GetAssetsPageStats	Assets page statistics	API Gateway

#	Function Directory	Purpose	Trigger
25	GetDashboardCounters	Pre-computed dashboard counters	API Gateway
26	GetDisposalDetails	Fetch disposal record details	API Gateway
27	GetEmployeeStats	Employee dashboard statistics	API Gateway
28	GetFinanceStats	Finance dashboard statistics	API Gateway
29	GetHandoverForm	Generate PDF handover form (Docker/WeasyPrint)	API Gateway
30	GetIssue	Fetch issue/repair record details	API Gateway
31	GetITAdminStats	IT Admin dashboard statistics	API Gateway
32	GetManagementStats	Management dashboard statistics	API Gateway
33	GetRecentActivity	Dashboard recent activity feed	API Gateway
34	GetRequestsEmployeeStats	Employee requests page statistics	API Gateway
35	GetRequestsITAdminStats	IT Admin requests page statistics	API Gateway
36	GetReturn	Fetch return record details	API Gateway
37	GetScanResults	Fetch Textract scan results with confidence	API Gateway
38	GetSignedHandoverForm	Fetch signed handover PDF	API Gateway
39	GetSoftwareRequest	Fetch software request details	API Gateway
40	InitiateAssetScan	Trigger Textract analysis on invoice	Internal
41	InitiateDisposal	IT Admin initiates asset disposal	API Gateway
42	InitiateReturn	IT Admin initiates asset return	API Gateway
43	ListAllIssues	List all issues across assets	API Gateway
44	ListAllReturns	List all returns across assets	API Gateway
45	ListAllSoftwareRequests	List all software requests	API Gateway
46	ListAssetCategories	List all asset categories	API Gateway
47	ListAssetDisposals	List disposals for specific asset	API Gateway
48	ListAssets	List all assets with pagination	API Gateway
49	ListDisposals	List all disposals	API Gateway
50	ListEmployeeSignatures	List signatures for an employee	API Gateway
51	ListEscalatedRequests	List escalated software requests	API Gateway
52	ListIssues	List issues for specific asset	API Gateway
53	ListMaintenanceHistory	List maintenance history for asset	API Gateway
54	ListMyAssets	Employee's assigned assets	API Gateway
55	ListMyIssues	Employee's submitted issues	API Gateway
56	ListMyNotifications	User's notifications	API Gateway
57	ListMySoftwareRequests	Employee's software requests	API Gateway
58	ListPendingDisposals	Management pending disposal approvals	API Gateway

#	Function Directory	Purpose	Trigger
59	ListPendingReplacements	Management pending replacement approvals	API Gateway
60	ListPendingReturns	List pending returns	API Gateway
61	ListPendingSignatures	Employee's pending handover signatures	API Gateway
62	ListReturns	List returns for specific asset	API Gateway
63	ListSoftwareRequests	List software requests for asset	API Gateway
64	ListUsers	List all users	API Gateway
65	MaintenanceStreamProcessor	Process maintenance history events	DynamoDB Stream
66	ManagementReviewDisposal	Management reviews disposal	API Gateway
67	ManagementReviewIssue	Management reviews replacement	API Gateway
68	ManagementReviewSoftwareRequest	Management reviews software request	API Gateway
69	MarkNotificationRead	Mark notification as read	API Gateway
70	NotificationProcessor	Create in-app notifications	DynamoDB Stream
71	ProcessDisposalEmailNotification	Send disposal status emails	SQS
72	ProcessEmailNotification	Unified email notification consumer	SQS
73	ProcessFinanceNotification	Send finance write-off emails	SQS
74	ReactivateUser	IT Admin reactivates user	API Gateway
75	RequestReplacement	IT Admin requests asset replacement	API Gateway
76	ResolveRepair	IT Admin resolves issue directly	API Gateway
77	ReviewSoftwareRequest	IT Admin reviews software request	API Gateway
78	ScanResultProcessor	Process Textract results	SNS
79	ScanWorker	Start Textract async analysis (Docker)	Internal
80	SendWarranty	IT Admin sends asset to warranty	API Gateway
81	SubmitAdminReturnEvidence	IT Admin submits return evidence	API Gateway
82	SubmitIssue	Employee submits issue report	API Gateway
83	SubmitSoftwareRequest	Employee submits software request	API Gateway
84	WebSocketConnect	WebSocket connection handler	WebSocket API
85	WebSocketDefault	WebSocket default route handler	WebSocket API
86	WebSocketDisconnect	WebSocket disconnection handler	WebSocket API

8.2 Error Handling Convention

All Lambda handlers follow a consistent error handling pattern:

Exception	HTTP Status	Source
`ValidationError` (Pydantic)	400	Invalid request body/params
`ValueError`	400	Business rule violation

Exception	HTTP Status	Source
`PermissionError`	403	Insufficient role/group
`NotFoundException`	404	Record not found
`ConflictException`	409	State conflict (locked record, duplicate)
`Exception` (unhandled)	500	Internal server error (logged)

8.3 Testing

- Framework: pytest
- AWS mocking: moto
- Test location: `tests/unit/`
- Each Lambda gets corresponding unit tests
- Shared fixtures in `conftest.py`

```
pytest tests/unit/
```

8.4 Scripts

Script	Purpose
`seed-dashboard-counters.py`	Initialize dashboard counter records
`seed-default-categories.py`	Seed default asset categories
`seed-employee-counters.py`	Initialize employee counter records
`seed-entity-counters.py`	Initialize entity counter records
`insert-super-admin.py`	Create initial super admin user
`delete-non-user-records.py`	Clean up non-user records
`delete-notification-records.py`	Clean up notification records
`fix-replacement-rejected-assets.py`	Fix assets stuck in rejected state
`migrate-ids-to-domain-prefix.py`	Migrate to domain-prefixed IDs
`migrate-issue-sk-to-uuid.py`	Migrate issue sort keys
`migrate-issue-status-to-troubleshooting.py`	Migrate issue status values
`migrate-records-to-uuid.py`	Migrate records to UUID format

End of Technical Design Document